

程序设计实习（实验班-2026春）

距离度量及其计算：高维欧氏距离

授课教师：姜少峰

助教：方心童、葛致远、李雷思问、刘鹏飞

Email: shaofeng.jiang@pku.edu.cn

Curse of Dimensionality

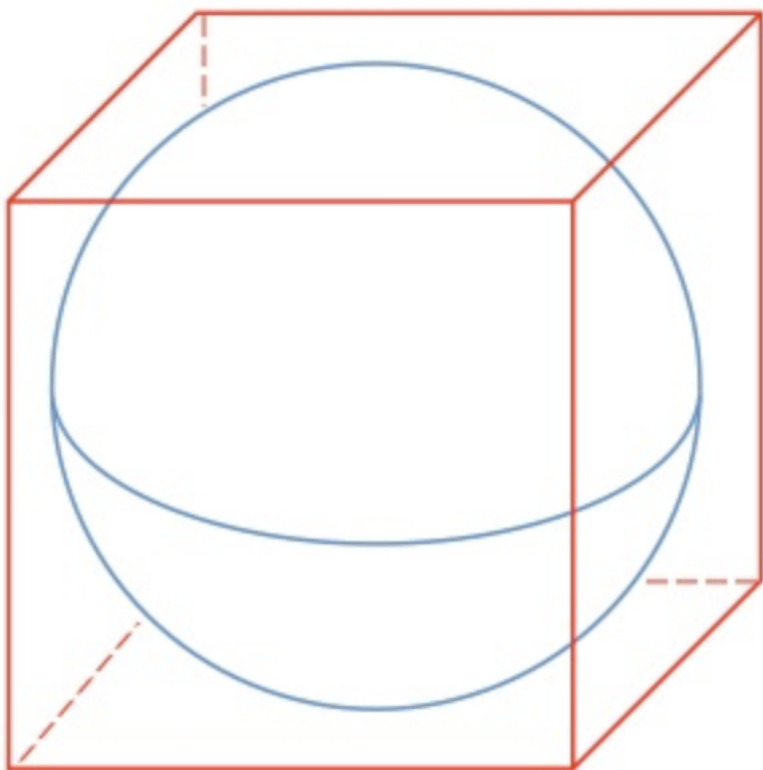
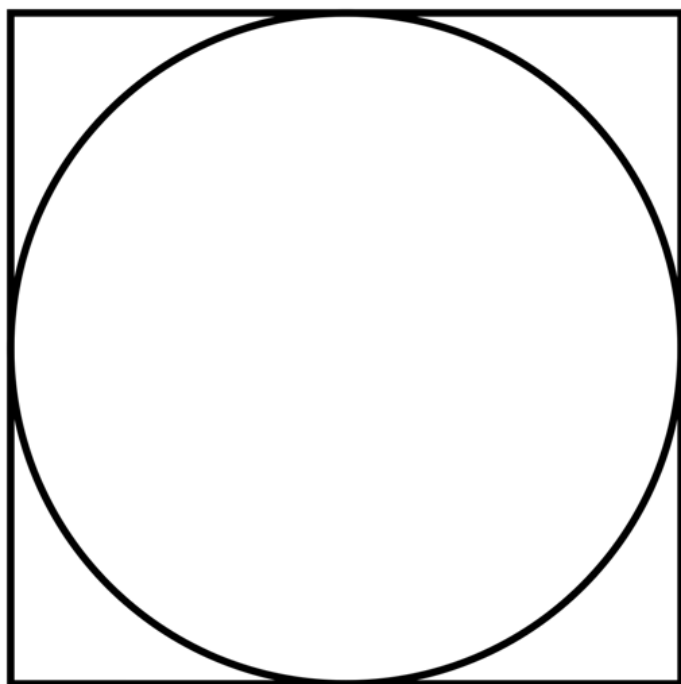
高维欧氏空间的“奇怪”特性

区别于低维空间上的常见直觉

- 高维空间还有很多其他“奇怪”的特性
- 方 vs 圆：单位立方体的体积是1，单位球的体积是 $O(2^{-O(d)})$

d维半径为r的球体积近似：
$$V_d(r) \approx \frac{1}{\sqrt{d\pi}} \left(\frac{2\pi e}{d} \right)^{d/2} \cdot r^d$$

- 另：单位圆和方的直径差 $\sqrt{d}$ 倍

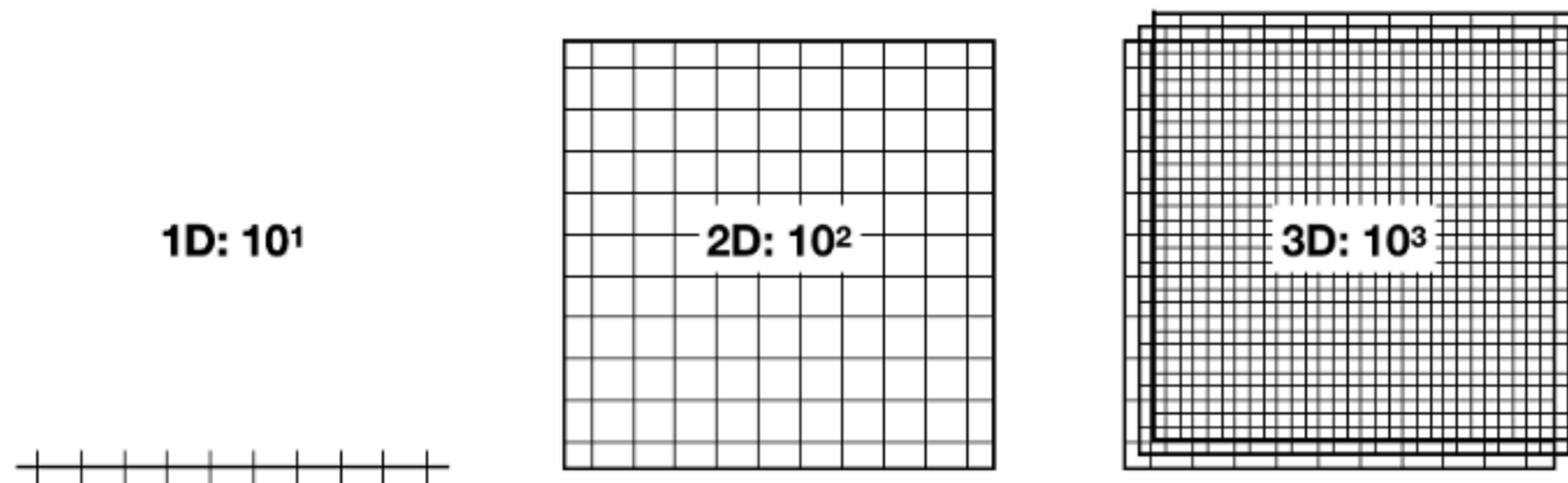


Dimension	Volume of a ball of radius R
0	1
1	$2R$
2	$\pi R^2 \approx 3.142 \times R^2$
3	$\frac{4\pi}{3} R^3 \approx 4.189 \times R^3$
4	$\frac{\pi^2}{2} R^4 \approx 4.935 \times R^4$
5	$\frac{8\pi^2}{15} R^5 \approx 5.264 \times R^5$
6	$\frac{\pi^3}{6} R^6 \approx 5.168 \times R^6$
7	$\frac{16\pi^3}{105} R^7 \approx 4.725 \times R^7$
8	$\frac{\pi^4}{24} R^8 \approx 4.059 \times R^8$
9	$\frac{32\pi^4}{945} R^9 \approx 3.299 \times R^9$
10	$\frac{\pi^5}{120} R^{10} \approx 2.550 \times R^{10}$
11	$\frac{64\pi^5}{10395} R^{11} \approx 1.884 \times R^{11}$
12	$\frac{\pi^6}{720} R^{12} \approx 1.335 \times R^{12}$
13	$\frac{128\pi^6}{135135} R^{13} \approx 0.911 \times R^{13}$
14	$\frac{\pi^7}{5040} R^{14} \approx 0.599 \times R^{14}$
15	$\frac{256\pi^7}{2027025} R^{15} \approx 0.381 \times R^{15}$
n	$V_n(R)$

Curse of Dimensionality

样本稀疏

- 考虑呈每一维坐标的均匀分布以及 n 个样本点 P
 - 有 2^d 个潜在样本，因此经常 $n \ll 2^d$
 - 这导致数据极为“稀疏”，很难形成对整个样本空间的有效覆盖/估计



Curse of Dimensionality

对于计算问题的挑战

- 关于 d 的依赖经常是指数级的!

需要在某些例如 $P \neq NP$ 的复杂度假设下

- 在若干计算模型和问题中, $d = O(\log n)$ 已经很难!

[Trevisan, SICOMP 00]

- 例如, 对于欧氏TSP, 无多项式时间 $(1 + \epsilon)$ -近似

- 对于最近点对和直径问题, 无 $n^{2-\delta}$ 时间 $(1 + \epsilon)$ -近似算法

[Karthic et al., ITCS 19]; [Chen, CCC 18]

- 对于数据流MST/FL, 无 $o(n)$ 空间 $(1 + \epsilon)$ 近似

[Chen et al., STOC 23]; [Czumaj et al., FOCS 22]

$d = \Theta(\log n)$ 已经是高维
很多问题高维下无 $(1 + \epsilon)$ 近似

2^d vs $\text{poly}(d)$

- 对于算法来说，关于 d 的依赖常常满足一个二分性：
 - 要么是 $\text{poly}(d)$ ，要么是至少 2^d
 - 尤其是 $2^{\log^2 d}$ 这种quasi-poly的很罕见
- 因此： $\text{poly}(d)$ 的是适用于高维的算法，而任意的 $f(d)$ 的则是适用于低维的

例如 2^{2^d} 也是可以接受的

- 另一种理解方式是：低维即 $d = O(1)$ ，高维即 $d = O(\log n)$

高维欧氏距离上的高效近似技术

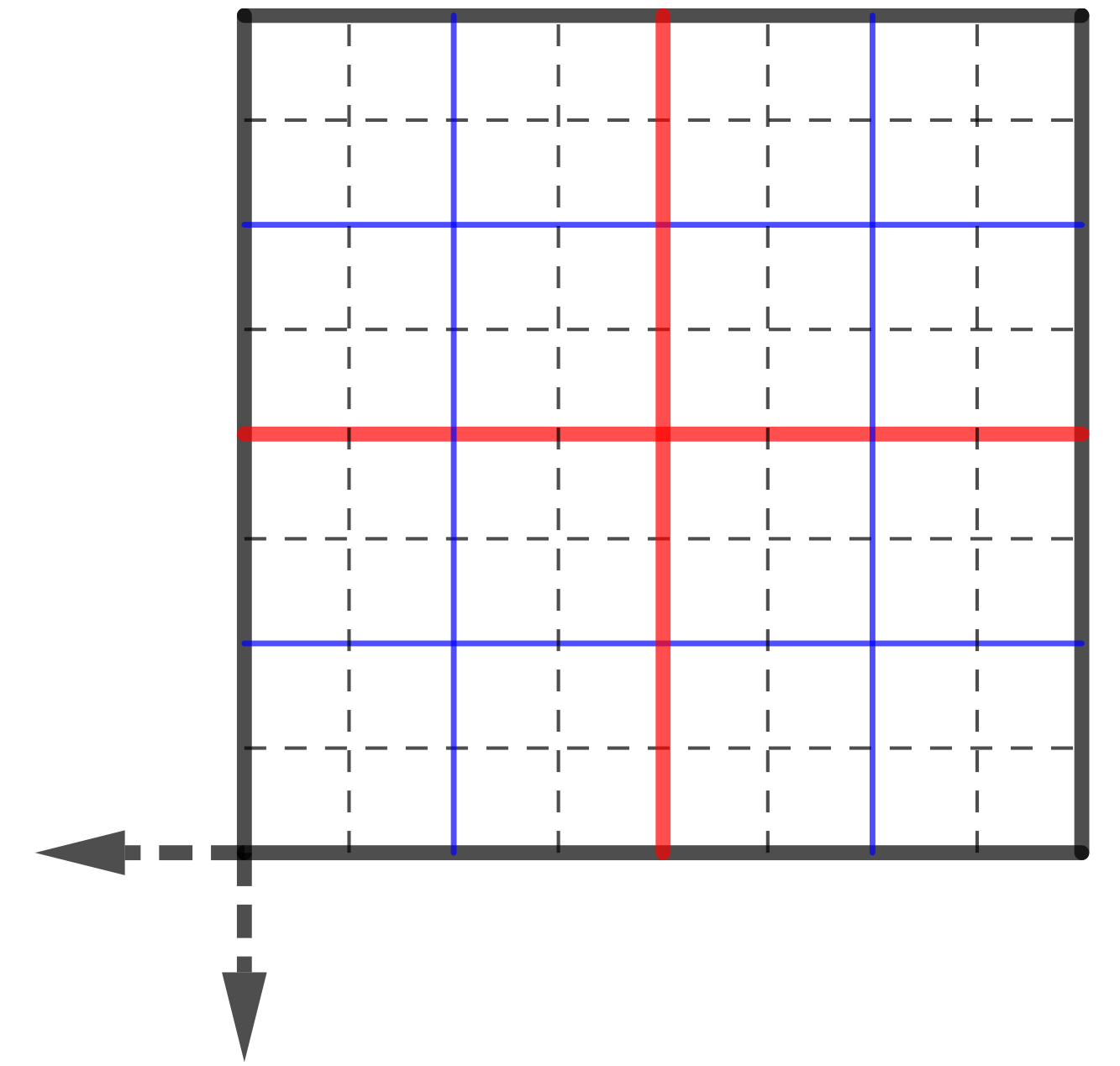
我们的侧重点

- 侧重运行快速、实现简便
- 代价是较大的（最坏情况下的）近似比，通常大于常数
- 本讲的近似比都与 d 有关，但后面的一讲会介绍如何能规约到 $d = O(\log n)$
 - 本讲及其作业并不需要该规约

一个通用技术： 随机平移四分树

每维独立生成 $[0, \Delta]$ 随机数

- 选取 $[0, \Delta]^d$ 上的均匀随机向量 v
- 构造四分树 $\text{BuildTree}([0, 2\Delta]^d, P)$ 后，将整个树平移 $-v$
 - 也可以看作 $\text{BuildTree}(\square = [-v, 2\Delta - v]^d, P)$
- 回忆：构造四分树复杂度为 $O(nd \log \Delta)$ ，不需要 2^d
- 我们将利用这同样的构造给出若干不同的应用方式



$[0, 2\Delta]$ 确保平移后依然覆盖点集

主要概率性质

称这种hypercube为 2^i -cube

- 考虑一对点 x, y , 以及边长为 2^i 的hypercube, 则

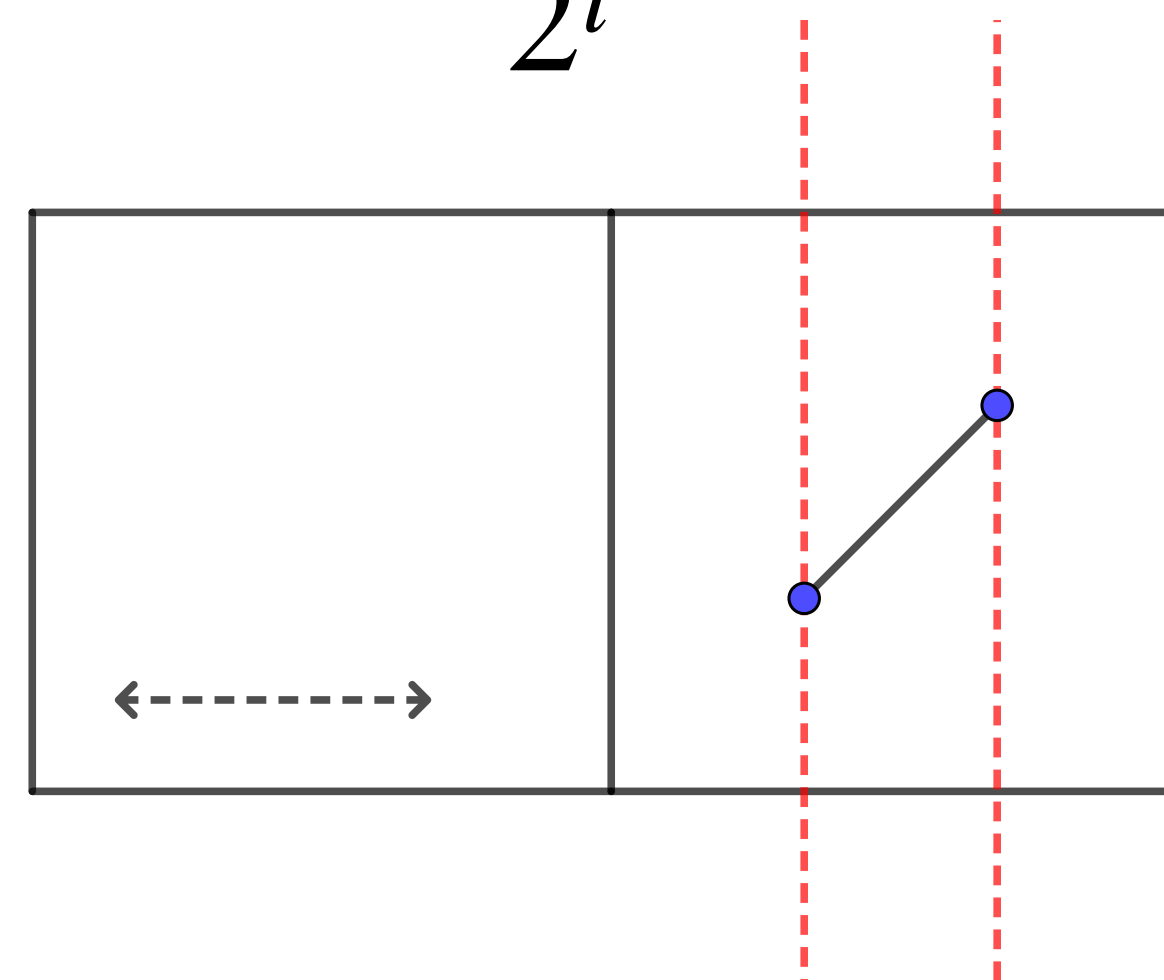
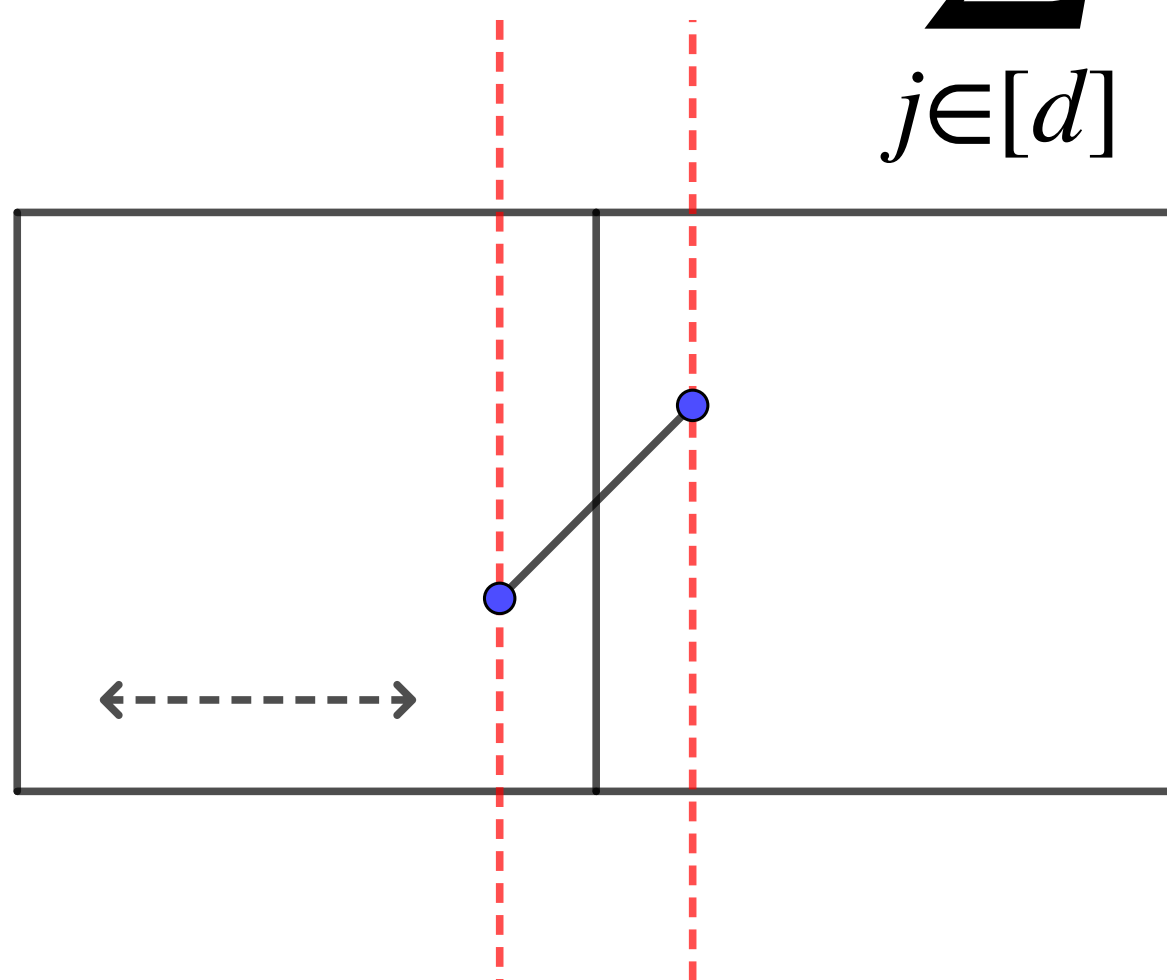
$$\Pr[x, y \text{ are cut by } 2^i\text{-cubes}] \leq \frac{\sqrt{d} \cdot \|x - y\|_2}{2^i}$$

这里cut的意思是说 x, y 不属于同一个 2^i -cube

- 直觉: “小边被大cube分开的概率很低”

主要概率性质

$$\begin{aligned}\Pr[x, y \text{ are cut by } 2^i\text{-cubes}] &= \Pr[\exists j \in [d], x, y \text{ are cut at coordinate } j \text{ of } 2^i\text{-cube}] \\ &\leq \sum_{j \in [d]} \Pr[x, y \text{ are cut at coordinate } j \text{ of } 2^i\text{-cube}] \\ &\leq \sum_{j \in [d]} |y_j - x_j| / 2^i \leq \frac{\sqrt{d} \|x - y\|_2}{2^i}\end{aligned}$$



1. 随机平移四分树与 Tree Embedding

Tree Embedding

- Tree embedding: 将数据集 P 从 $\mathbb{R}^d$ 映射到一棵树 $T(V, E)$ 上, 其中 $P \subseteq V$
 - 旨在用树上的距离来“近似”/“代替”欧氏距离
 - 首先要求 $\forall x, y \in P, \text{dist}_T(x, y) \geq \|x - y\|$
- 主要性能衡量叫distortion:

是 ≥ 1 的量, 越小越好

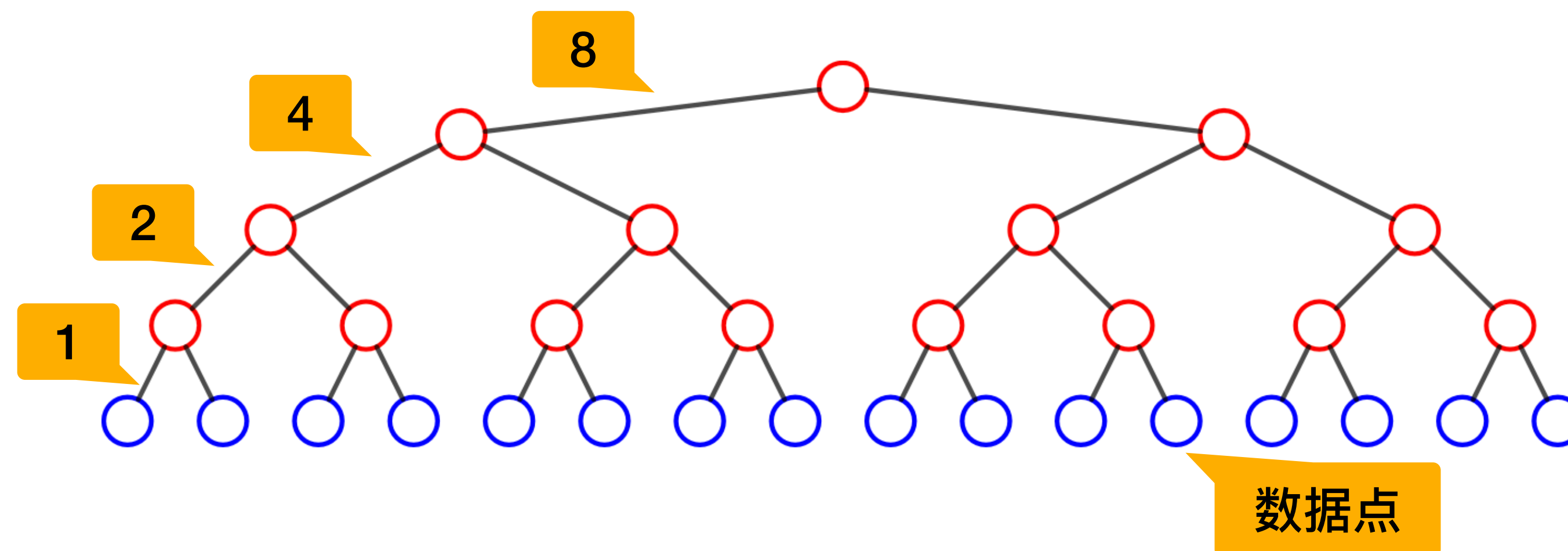
$$\max_{x \neq y \in P} \frac{\text{dist}_T(x, y)}{\|x - y\|}$$

Tree Embedding额外要求: 2-HST

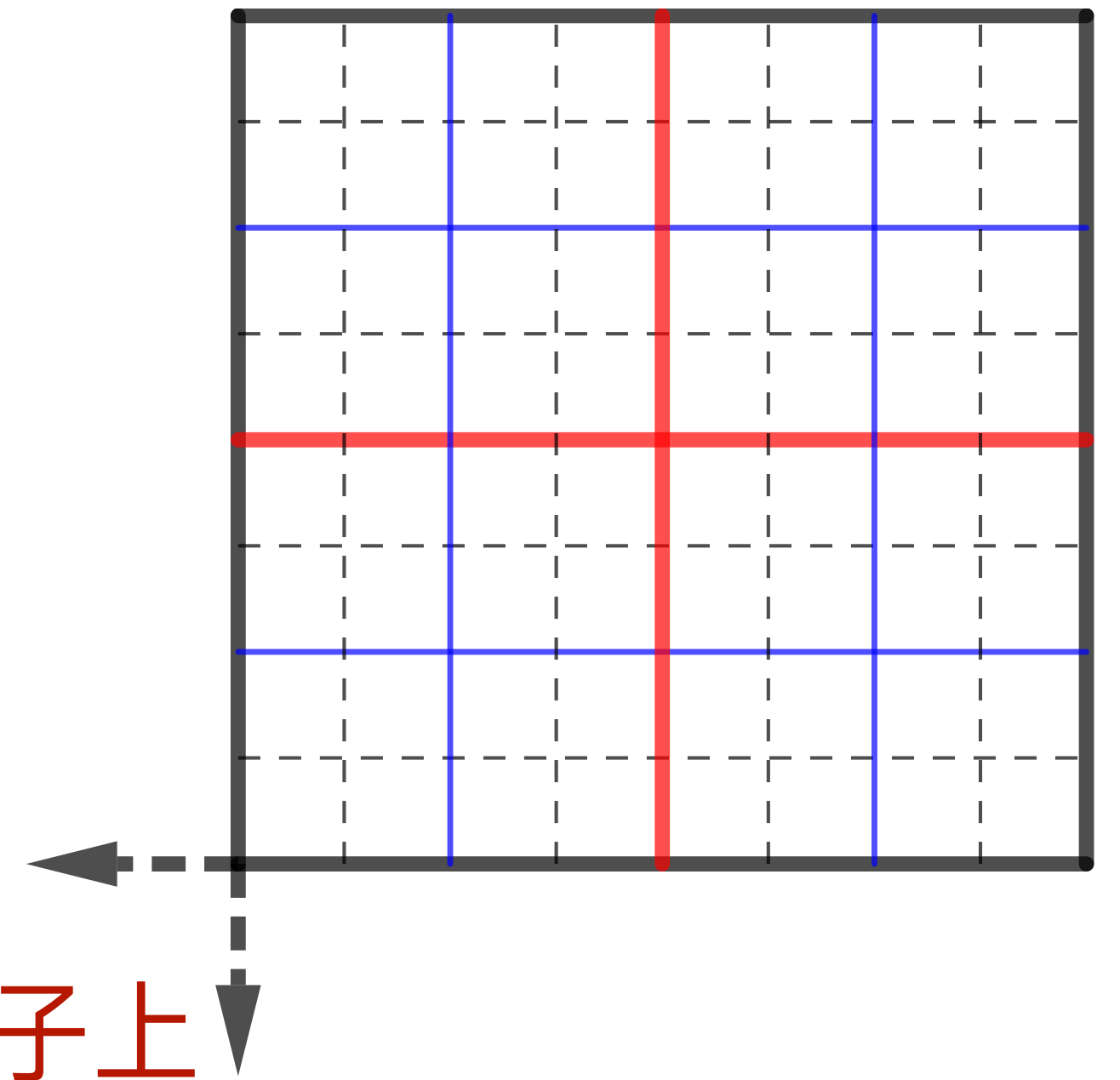
HST = hierarchically separated tree

一般要求这个树有更好的结构:

- 数据集P都在树的叶子上, 且所有叶子高度相同 (所以内部点都只是辅助点)
- 考虑任何一条从叶子到根的路径, 边权依次乘以2



构造算法和结论



- 首先构造随机平移四分树
- 定义树 T : 每个 $\square$ 定义一个树节点, 数据点都放在树的叶子上
- 定义边: 每个 $\square$ 连接到所有的直接孩子, 赋予 $\sqrt{d} \cdot 2^i$ 的权值, 设 $\square$ 边长 2^i
- 对于 $x, y \in P$ (都是树的叶子), 距离 $\text{dist}_T(x, y)$ 就是树上的距离
- 结论: $\forall x, y \in P, \text{dist}_T(x, y) \geq \|x - y\|, \mathbb{E}[\text{dist}_T(x, y)] \leq O(dh) \cdot \|x - y\|$

下界是确定性的

上界是随机的

h 是四分树的高度

关于距离保证

- $\mathbb{E}[\text{dist}_T(x, y)] \leq O(dh) \cdot \|x - y\|$ 的保证只是数学期望
 - 因此如果考虑一个具体的T，很可能很多对点距离并不能达到该保证
- 可以证明，确定性/高概率保证一般而言是不可能的
 - 因此随机性对于tree embedding来说有本质的必要性
- 最优的distortion：（需要采用不同的算法）可以达到 $\Theta(\log n)$ 的distortion
 - Fakcharoenphol, Rao, Talwar: A tight bound on approximating arbitrary metrics by tree metrics. JCSS (2004)

目前已知实现运行时间较差

扩展到其他设定

- 我们描述的算法可以在不读入具体数据集的情况下运行
 - 我们只是在 $[\Delta]^d$ 上运行，可以把所有点都当叶子看待
 - 虽然实际数据集只是这些叶子的子集，但多余的叶子对于distortion并无影响
- 并且输出可以用查询的方式按需给出：
 - 给定一个数据点 x ，算法输出 x 从叶子到树根路径上所有树节点的ID
 - 应用：可以在数据流等空间受限设定下不显式建树，达到极小的空间占用

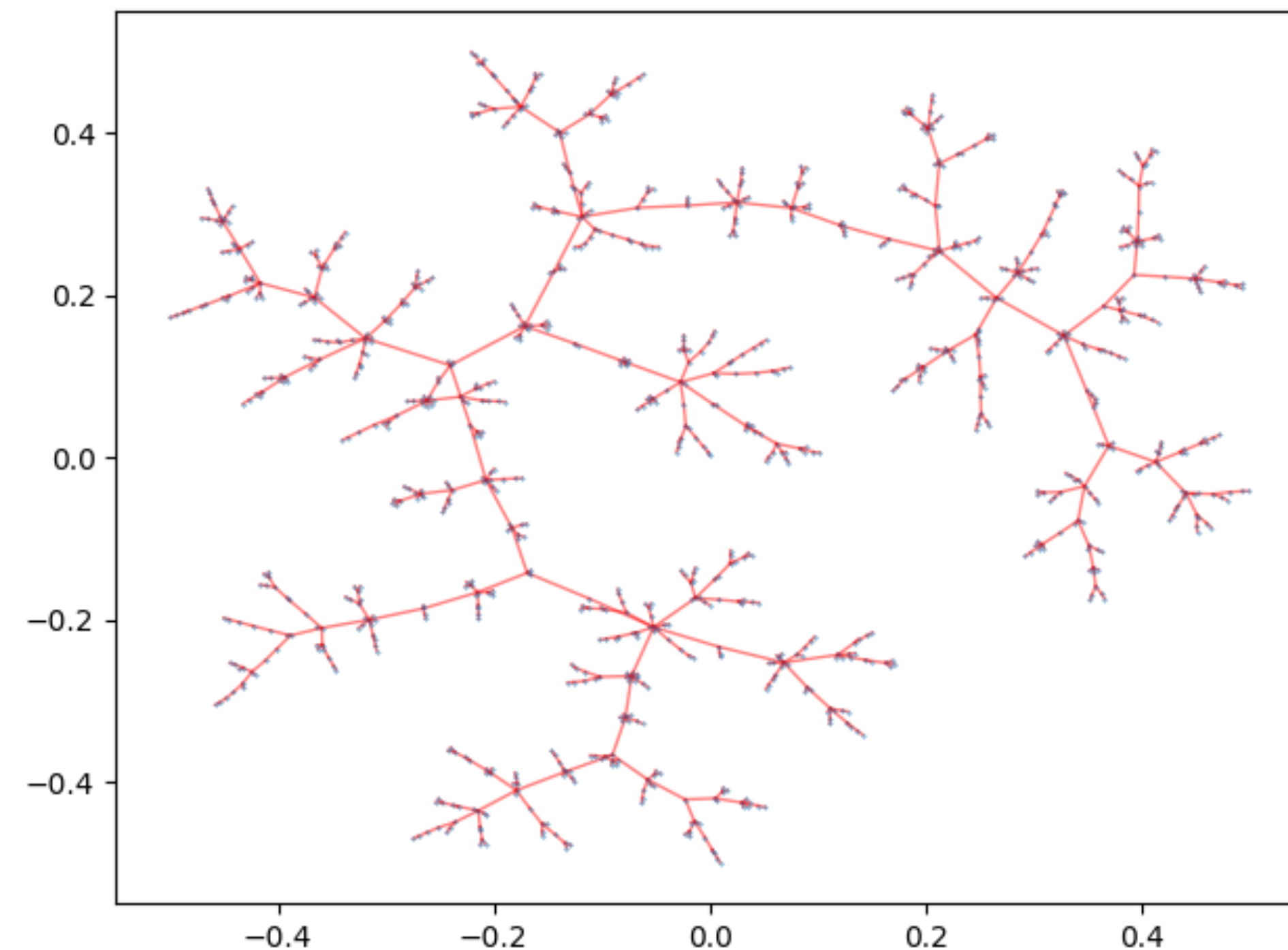
数据流算法

- 该tree embedding可以实现成数据流算法
 - 在数据流开始前采样平移量 v
 - 之后对每个数据点 x 计算 $x + v$
 - 可用 $O(d)$ 空间时间计算所处的格子及其所有祖先

应用：高维欧氏最小生成树

复习定义

- 图的最小生成树（MST）：最小权的使所有点连通的子图
- 欧氏数据点集 P 的最小生成树：定义在 P 的欧氏图上的MST



Tree Embedding近似高维MST

$O(dh)$ 近似比

- 结论: $\forall x, y \in P, \text{dist}_T(x, y) \geq \|x - y\|, \mathbb{E}[\text{dist}_T(x, y)] \leq O(dh) \cdot \|x - y\|$

- 高维MST算法:

如何找 F_T ?

- 先建立 T , 然后在 T 上找 (所有叶子的) MST, 记为 F_T , 并设精确MST解为 F^*

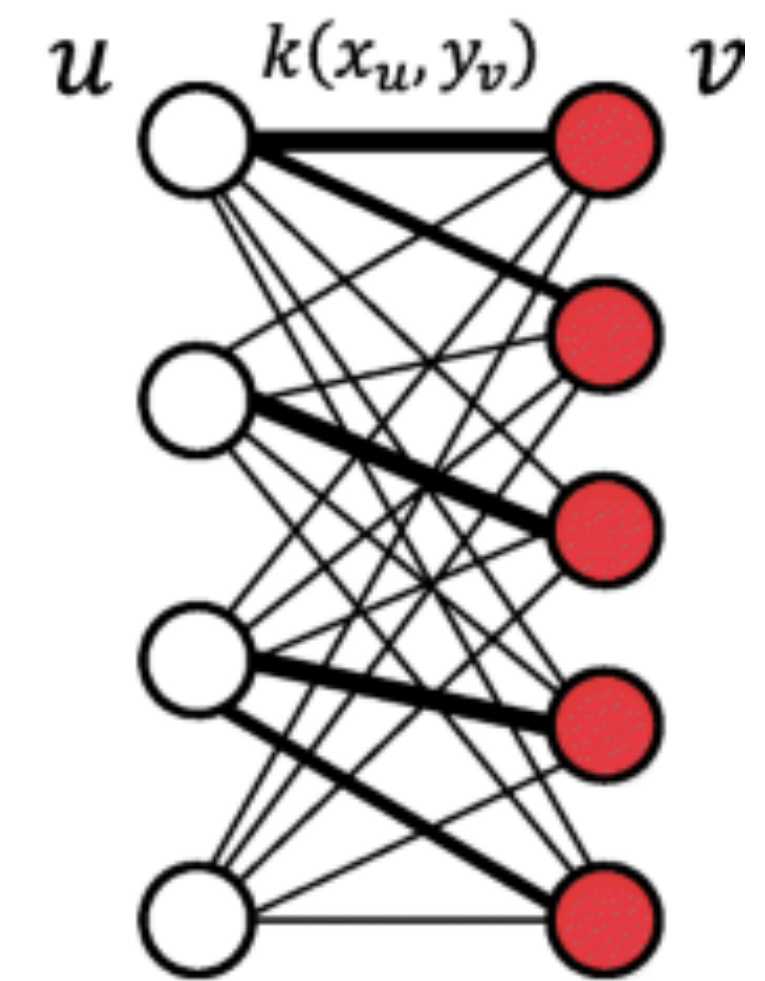
观察到 $w(F_T) \leq \sum_{x, y \in F^*} \text{dist}_T(x, y)$, 所以

注意: F_T 并不是 P 的欧氏图的子图, 因此并不是 P 的生成树;
如何将 F_T 转化成 P 上的生成树而不增加代价?

$$\mathbb{E}_T[w(F_T)] \leq \mathbb{E}\left[\sum_{x, y \in F^*} \text{dist}_T(x, y)\right] \leq O(dh) \sum_{x, y} \|x - y\| = O(dh) \cdot w(F^*)$$

其他应用：高维欧氏最小权匹配

- 最小权匹配问题： $2n$ 个 $\mathbb{R}^d$ 上的点的最小欧氏权值的匹配
 - 匹配： n 条边的集合，使得每个点参与到恰好一条边里
 - 权重： 边 $\{x, y\}$ 的权重是 $\|x - y\|_2$ ，优化的是边权和
- 相关问题： 二分/双色匹配（这里没有这条限制）



匹配与Tree Embedding

- 性质：在tree embedding上，子树内的点应该优先与同一子树的匹配
- 因此：仅当一个子树有奇数个后继时，需要将其跨子树匹配
 - 注意到：所有数据点都在叶子上，所以具体用哪个点匹配并不重要
- 如何实现成算法？

作业十五：Tree Embedding近似高维欧氏最小权匹配

- <http://cssyb.openjudge.cn/26hw15/>
- Deadline: x月x日
- 分值：2分

对比

- 劣势：近似比大 $O(dh)$ ，尤其是达不到常数近似比
- 优势：
 - 最小生成树、聚类、匹配等各类问题；
很多NP-hard问题在树上仍有有效算法
 - 因为每对点期望距离都能保持，因此广泛适用于目标函数是点距离和的问题
 - 树上算法性能较好，可解决问题多，有树算法就有欧氏空间近似算法、
 - 没有 2^d ，对于高维也适用
 - 可以扩展到数据流等设定

Tree Embedding分析

树上距离的表示、下界

x和y的树上距离：考虑最小的同时包含x, y的□

对一路上的边权求和

Fact: 若LCA(x, y)的□边长 2^i , 那么树上距离是 $2 \cdot \sqrt{d} \sum_{j=0}^i 2^j = \Theta(\sqrt{d} \cdot 2^i)$

Claim: $\text{dist}_T(x, y) \geq \|x - y\|$

验证：设LCA(x, y)的□边长 2^i , 那么 $\|x - y\| \leq \sqrt{d} 2^i \leq \sum_{j=0}^i \sqrt{d} 2^j$

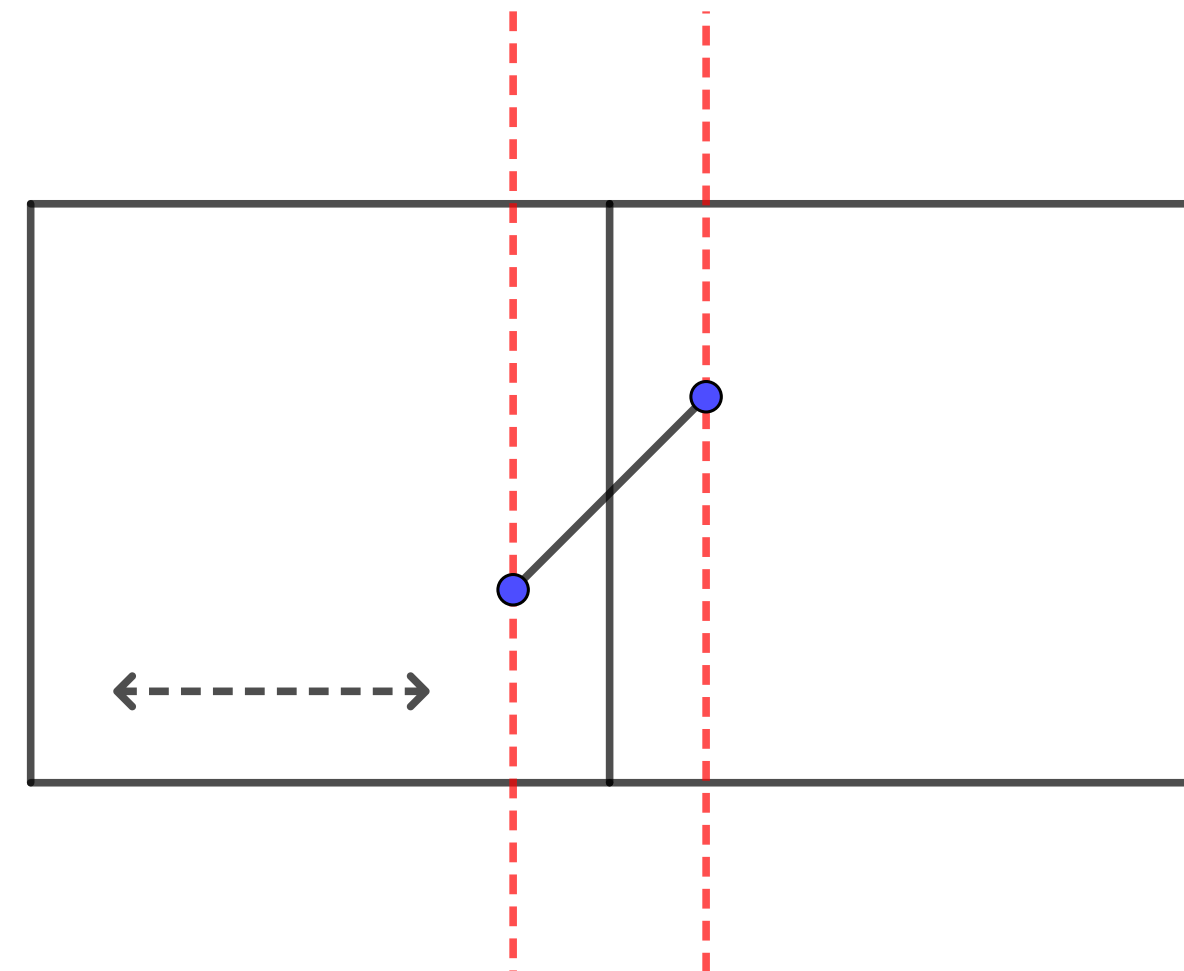
上界

验证: $\mathbb{E}[\text{dist}_T(x, y)] \leq dh \cdot \|x - y\|$

- 回忆: 考虑 x, y , 设 $\text{LCA}(x, y)$ 的 $\square$ 边长 2^i , 则

$$\text{dist}_T(x, y) = 2 \cdot \sqrt{d} \sum_{j=0}^i 2^j = \Theta(\sqrt{d} \cdot 2^i)$$

$$\begin{aligned} \mathbb{E}[\text{dist}_T(x, y)] &= \sum_{j=0}^h \Pr[\text{LCA is at } 2^j\text{-cube}] \cdot \Theta(\sqrt{d} 2^j) \\ &\leq \sum_{j=0}^h \Pr[x, y \text{ cut by } 2^{j+1}\text{-cubes}] \cdot O(\sqrt{d} 2^j) \\ &\leq O(hd) \cdot \|x - y\| \end{aligned}$$



$$\Pr[x, y \text{ are cut by } 2^i\text{-cubes}] \leq \frac{\sqrt{d} \cdot \|x - y\|_2}{2^i}$$

2. 随机平移四分树与 Spanner

高维Spanner

复习定义

$$t \geq 1$$

- t -Spanner是数据集 $P \subseteq [\Delta]^d$ 的一个欧氏图的子图 H
 - 欧氏图：点集还是 P ，边集是 $P \times P$ ，边权重是端点间的欧氏距离
 - 子图：点集相同，边集是子集
 - t -Stretch：要求对任何 $x, y \in P$ 有 $\|x - y\| \leq \text{dist}_H(x, y) \leq t \cdot \|x - y\|$
 - 追求：给定 t ， H 满足 t -stretch 条件，且边数尽量少 dist_H 代表 H 上的最短路距离
- 可看作是欧氏图的稀疏“压缩”，后续可使用稀疏图上的算法来解决欧氏问题

高维Spanner结论与算法

- 结论：存在一个 $O(d)$ -spanner，构造时间/边数 $O(n \log(n) \log(\Delta))$
- 算法：初始化spanner边集 $H \leftarrow \emptyset$
 - 设 $m \leftarrow O(\log n)$ ，独立采样 m 个随机平移四分树 $T_1, \dots, T_m$
 - 对每个 $j \in [m]$ ，每个树上的节点 $u \in T_j$
 - 设 P_u 为 u 含有的数据点集合，将 P_u 上的任意一个星形图加入 H
- 边数、时空复杂度的分析？

星形图：除了一个树根外其余全是叶子的树

作业十六：高维Spanner近似MST

- <http://cssyb.openjudge.cn/26hw16/>
- Deadline: x月x日
- 分值：3分
- 潜在的也可尝试tree embedding，但实际近似比可能略差（标程采用spanner）

分析

$$\Pr[x, y \text{ are cut by } 2^i\text{-cubes}] \leq \frac{\sqrt{d} \cdot \|x - y\|_2}{2^i}$$

- 考虑任意的数据点 x, y
- 我们需要spanner有一条 $x \rightsquigarrow y$ 长度至多 $O(d) \cdot \|x - y\|_2$ 路径
- 设 i_{xy} 满足 $2^{i_{xy}} \approx \|x - y\|_2$
- 若存在 T_j 使 x, y 同属于某 $O(\sqrt{d}) \cdot 2^{i_{xy}}$ -cube, 则已经找到了这样的路径
- 考虑反面: 所有 m 个 T_j 都在 $2^{i_{xy}}$ -cube 上分开 x, y , 概率至多是 $2^{-m} = 1/\text{poly}(n)$
- union bound后可以得出所有点对都满足 $O(d)$ 的误差

星形图的直径至多是 $2\sqrt{d}$ 倍的cube直径

3. 随机平移四分树与 Range Query

Range Query问题

- 输入：在 $\mathbb{R}^d$ 中的 n 个数据点集合 P ，参数 $r > 0$
- 查询：给定 $q \in \mathbb{R}^d$ ，估计 $|B(q, r) \cap P|$ 的点数
- 一般而言： 查询要求估计 $f(B(q, r) \cap P)$ ，其中 f 是composable函数
 - 称一个函数 $f: 2^P \rightarrow \mathbb{R}$ 是composable的，若对任意的 S 及其划分 $S_1, \dots, S_t$
 $f(S)$ 可以从 $f(S_1), \dots, f(S_t)$ 的值推出
 - 例如sum、max、min等

近似保证的定义、结论

- 引入记号 $B_P(q, r) := B(q, r) \cap P$
- 这里考虑下面这种 α -近似: 实际返回的 E 满足 $|B_P(q, r)| \leq E \leq |B_P(q, \alpha r)|$

即类似于近似近邻查询的要求, 允许将 αr 内的点计入

- 我们介绍一种基于随机平移网格的算法

只需要网格, 不需要四分树,
但必须随机平移

- 查询达到 $O(\alpha)$ -近似、期望用时 $O(2^{d/\alpha^{2/3}} \text{poly}(d))$

主要针对大近似比

即牺牲精度改进指数上的依赖
选取足够大的 α 可以做到 $d^{1.5}$ -近似, $\text{poly}(d)$ 时间
这个 $\alpha^{2/3}$ 不一定是紧的, 只是目前的分析能达到的界

算法

- 预处理：作边长 $\ell = \alpha r / \sqrt{d}$ 的网格划分，并作随机平移（平移量每维在 $[0, \ell]$ ）
- 找到所有使得 $\square \cap B(q, r) \neq \emptyset$ 的 $\square$ ，记这个 $\square$ 形成的集合是 Q
- 返回 $\sum_{\square \in Q} |\square \cap P|$ ，即与 Q 中 $\square$ 相交的数据点数，作为最终的估计
- 正确性： $B(q, r)$ 内的点都计入了；至多计入 $B(q, (1 + \alpha)r)$ 的点
- 复杂性：可以证明 $|Q| \leq 2^{d/\alpha^{2/3}} \text{poly}(d)$

作业十七：高维Range Query

- <http://cssyb.openjudge.cn/26hw17>
- Deadline: x月 x日
- 分值：4分
- 标程参数选取： $\ell = 3r$ ，近似比最坏是 $3\sqrt{d} + 1$

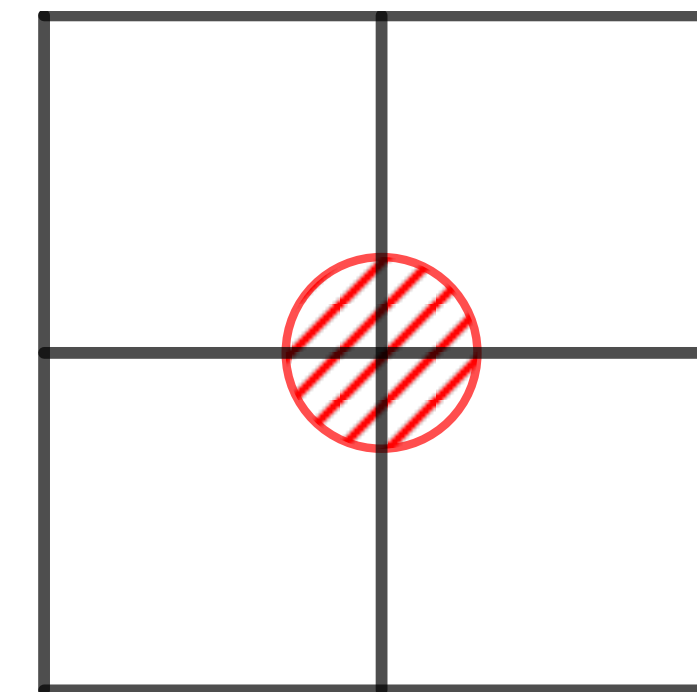
实现细节讨论

```
void search(int i, int id[]) {  
    if (i == d) {  
        if (dist(q, sq(id)) <= r) Q.add(id);  
        return;  
    }  
    id[i] = id_q[i]; search(i + 1, id);  
    id[i] = id_q[i] - 3r; search(i + 1, id);  
    id[i] = id_q[i] + 3r; search(i + 1, id);  
}
```

sq(id)即对应于id的□

id_q是q点所在□的id

- 以标程 $\ell = 3r$ 的设定为例
- 关键步骤：求 Q ，即 $\square \cap B(q, r) \neq \emptyset$ 的□集合
- 暴力找 Q ：从 q 所在的□开始，探索相邻的格子
 - 设所有□都以最小坐标的顶点坐标作为ID
 - 从 q 所在□的ID向量的每一维，依次进行DFS搜索，穷举{不变, $+3r$, $-3r$ }
- 保留到 q 距离在 r 内的□



优化

- 完全DFS暴力搜索至少需要 2^d

- 剪枝：让算法需要考虑的id数组至多有 $O(|Q| \cdot d)$ 个

- 设DFS运行到某个维度 $i \in [d]$ ，并设目前穷举到的是id[]

- 如果此时的sq(id)已经距离 q 大于 r 了，那后面 $d - i$ 维不用继续考虑，可以剪枝

- 原因：后面 $d - i$ 维都不改变时已经距离大于 r 了，改变后距离只会更大

- 可选优化：Q需要实现id（即d维数组/字符串）的集合，可以考虑Trie

```
void search(int i, int id[]) {  
    if (i == d) {  
        if (dist(q, sq(id)) <= r) Q.add(id);  
        return;  
    }  
    if (dist(q, sq(id)) > r) return;  
    id[i] = id_q[i]; search(i + 1, id);  
    id[i] = id_q[i] - 3r; search(i + 1, id);  
    id[i] = id_q[i] + 3r; search(i + 1, id);  
}
```

可选：用Trie实现Q

剪枝

剪枝的效果

具体指相差 $3r$ （因为我们仅考虑算法产生的id）

- 考虑一个辅助图：每个id对应一个vertex，两个id连边当且仅当恰有一维不同
 - 我们的search算法可以看作是在该图上做DFS，起点就是查询点 q 的id
 - 因此每个vertex的degree是 d
 - Q 里面的点对应于一个连通分量，记作 W
- 剪枝相当于：在DFS遇到连通分量 W 外面的第一个点时停止继续DFS
 - degree是 d ，所以最多额外扩展了 d 倍，也就是 $d|Q|$